

ToolX EdgeSync

Team Name: XRunners

Yuvraj Dhadwal, Patrick Huynh, Minjae Lee, Arthur Mahu, Christian Vela

Client: Yassine Bennani, Alha Kane

Repository: <https://github.com/yuvrajdhadwal/ToolX-EdgeSync-DCA-6101>

Table of Contents

Table of Contents.....	2
Table of Figures	2
Terminology.....	4
Introduction.....	5
Background	5
Document Summary	5
Static System Architecture.....	5
Introduction.....	5
Rationale.....	5
Static Architecture	6
Data Design	7
Introduction.....	7
Enhanced Entity Relationship Diagram	8
File Use	9
Data Exchange.....	9
Security Considerations	9
Dynamic System Architecture	10
UI Design and Heuristic Analysis	10
Developer.....	10
Developer Manager	12
Business Manager.....	15
Field Professional	26
Appendix	27

Table of Figures

Figure 1 UML of Static System Architecture	7
Figure 2 Entity Relationship Diagram.....	8

Figure 3 Developer: Firmware Page	10
Figure 4 Developer: Upload New Firmware	11
Figure 5 Developer Manager: Firmware Details	12
Figure 6 Developer Manager: Approve Pending Firmware.....	13
Figure 7 Developer Manager: Reject Pending Firmware.....	14
Figure 8 Business Manager: Home Page	15
Figure 9 Business Manager: World Map	16
Figure 10 Business Manager: World Map Panel	17
Figure 11 Business Manager: Add Shop.....	18
Figure 12 Business Manager: Devices Page	19
Figure 13 Business Manager: Device Information	20
Figure 14 Business Manager: Deployment History.....	21
Figure 15 Business Manager: Remove Device	22
Figure 16 Business Manager: Add New Device	23
Figure 17 Business Manager: Firmware Page	24
Figure 18 Business Manager: Deploy Firmware	25
Figure 19 Field Professional: View Devices	26
Figure 20 Field Professional: Device information.....	27

Terminology

API: an Application Programming Interface (API) is a defined set of rules that allow different software systems to communicate with each other.

Azure BLOB Storage: an object storage solution for the cloud, designed to store massive amounts of unstructured data such as text, images, videos, backups, and logs. It is highly scalable, durable, and accessible.

Azure IoT: Infrastructure that provides the connectivity, processing, and management capabilities needed to integrate physical devices with cloud and edge services

Cloud-Native: a system that takes advantage of cloud resources and development operations. This is beneficial since cloud services can be scaled from 0 to 100 with minimal effort from the developer.

Edge Devices: hardware components that are entry points to a network. They process data near the source before sending data packets to central servers or cloud systems for further computation.

Fast API: a modern Python web framework used to quickly build high-performance REST APIs with automatic documentation support.

JWT: JSON Web Token is a compact, URL-safe token format used to securely transmit verified user identity and claims between parties, typically for authentication and authorization.

Linux: a free and open-source operating system kernel.

Node.js: an open-source, cross-platform JavaScript runtime environment that allows developers to run JavaScript on the server side.

OTA: Over-The-Air updates are a method of remotely delivering software or firmware updates to devices via a network connection.

React.ts: a TypeScript file used in React applications that combines React's component-based UI library with TypeScript's static typing for safer and more scalable frontend development.

SQLite: a lightweight, file-based relational database system that stores data locally without requiring a separate server.

TLS: Transport Layer Security is a cryptographic protocol that encrypts data in transit to provide secure communication over a network, such as HTTPS.

Introduction

Background

We will design and implement a simplistic yet scalable software infrastructure for managing over-the-air (OTA) firmware updates across industrial edge devices running embedded Linux. The system will streamline update deployment, ensure end-to-end security, and support future growth with modular architecture and cloud-native components.

Document Summary

The System Architecture section describes the static components of our system's backend and frontend. The section includes the rationale behind the decisions of architectural design that the team made. The static system architecture diagram displays architectural components and logical relationships between each component.

The Data Design section describes the database structure of our application. We provide an Enhanced Entity Relationship Diagram for the SQLite database. Additionally, this section will also cover subsections for firmware file usage, data exchanges between edge devices and the IoT Hub, as well as for any security considerations involved.

Static System Architecture

Introduction

With this system we hope to establish these core functionalities: The workflow for the acceptance and declining of firmware, developed by the developers, by the developer manager, the workflow of deployment of firmware that comprises of the business manager approving firmware sent from the developer manager, the workflow of the field technician either accepting or denying the firmware and the installation and tracking of the firmware's history and other metadata.

Rationale

The main architectural decision was figuring out how to orchestrate communication between our web server and the thousands of globally located edge devices. Our main consideration for this was utilizing existing technology that our clients were familiar with. SLB uses Azure IoT as their current solution, which is why we are stuck with that.

For our frontend and backend, we chose to use React and Python FastAPI since these were the technologies we were most familiar with. For our database, we chose SQLite since we wanted a relational database that was lightweight.

We wanted to host on the cloud since we planned to make our infrastructure easily scalable. We chose Azure, since we were already within their ecosystem with Azure IoT.

For our edge device code, we went with C++ since there is an Azure IoT SDK for C. C++ has low memory overhead with easy access to the system calls that Linux provides, which was perfect for working with the edge agent. We chose not to go with C since C++ is a superset of C, so we get everything C has with more.

For our team's security considerations, we decided to use JWT authentication within our FastAPI backend because JWTs are stateless, meaning the client server can verify the user's identity without having to store their session data into our backend. JWT uses asymmetric signing using public/private key pairs to ensure the integrity of the token, allowing the client to be able to verify the sender's identity and ensure that the data isn't altered in the backend. Additionally, JWTs use short-lived expiration tokens to mitigate token theft. Besides from JWTs, we will also need to implement network security measures such as Transport Layer Security (TLS) to ensure data communication over a network is secured and encrypted.

Static Architecture

All users interact with our system through the frontend layer. At this layer, they can view all the possible firmware versions, devices, and any information they need for their role. The frontend is implemented with Typescript and Node.js. Field/Shop Professionals will also have the functionality of interacting with a device shell. From this shell, they can communicate to the system that their device is ready to install a deployed firmware. These devices have a C++ state machine that is able to handle all the logic necessary for our application. The Webapp requests and posts data to a backend API backed by Python FastAPI. This API handles all the business logic for the webapp, including storing persistent data in a SQLite Database: including but not limited to device location, current device version, and different kinds of firmware. To store the actual binary firmware that is to be installed on devices, we elected to use Azure BLOB Storage since it is designed to store large files easily while SQLite is meant to be lightweight. To handle the orchestration between this Python backend and thousands of devices globally, we utilize Azure IoT. Our Azure IoT handles communication and networking between the devices, Azure BLOB, and the Python backend. This is illustrated in Figure 1.

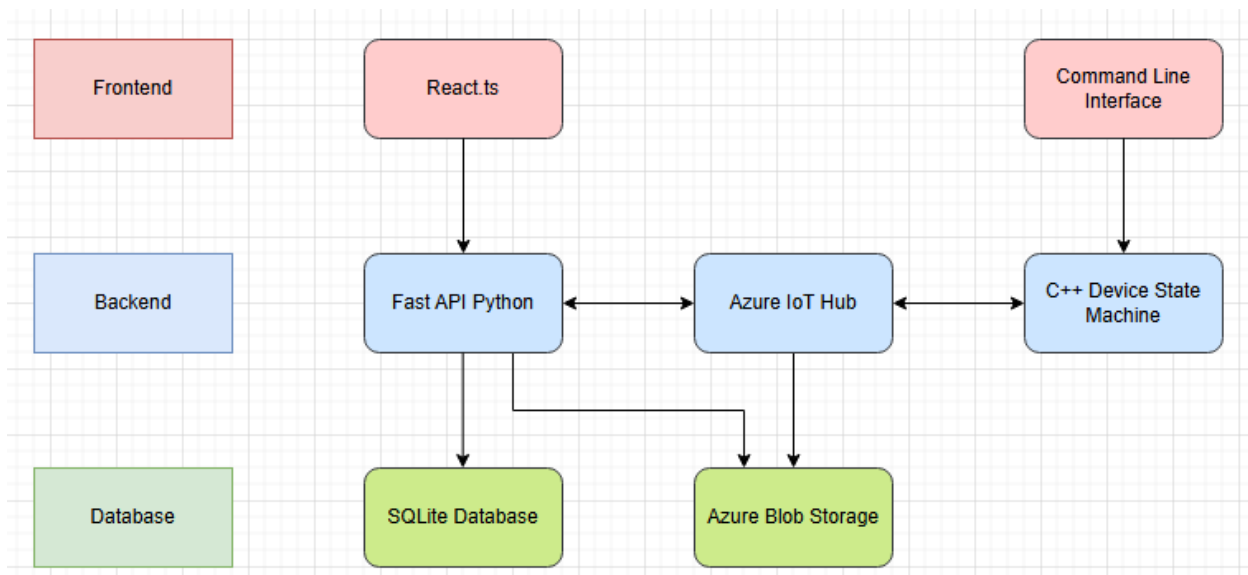


Figure 1 UML of Static System Architecture

Data Design

Introduction

Figure 2 is an Enhanced Entity Relationship diagram representing our application. The diagram shows the types of entities, attributes, and relationships between entities. The type of database of the project is SQLite. After the diagram, file usage, data exchange, and data security considerations are discussed.

Enhanced Entity Relationship Diagram

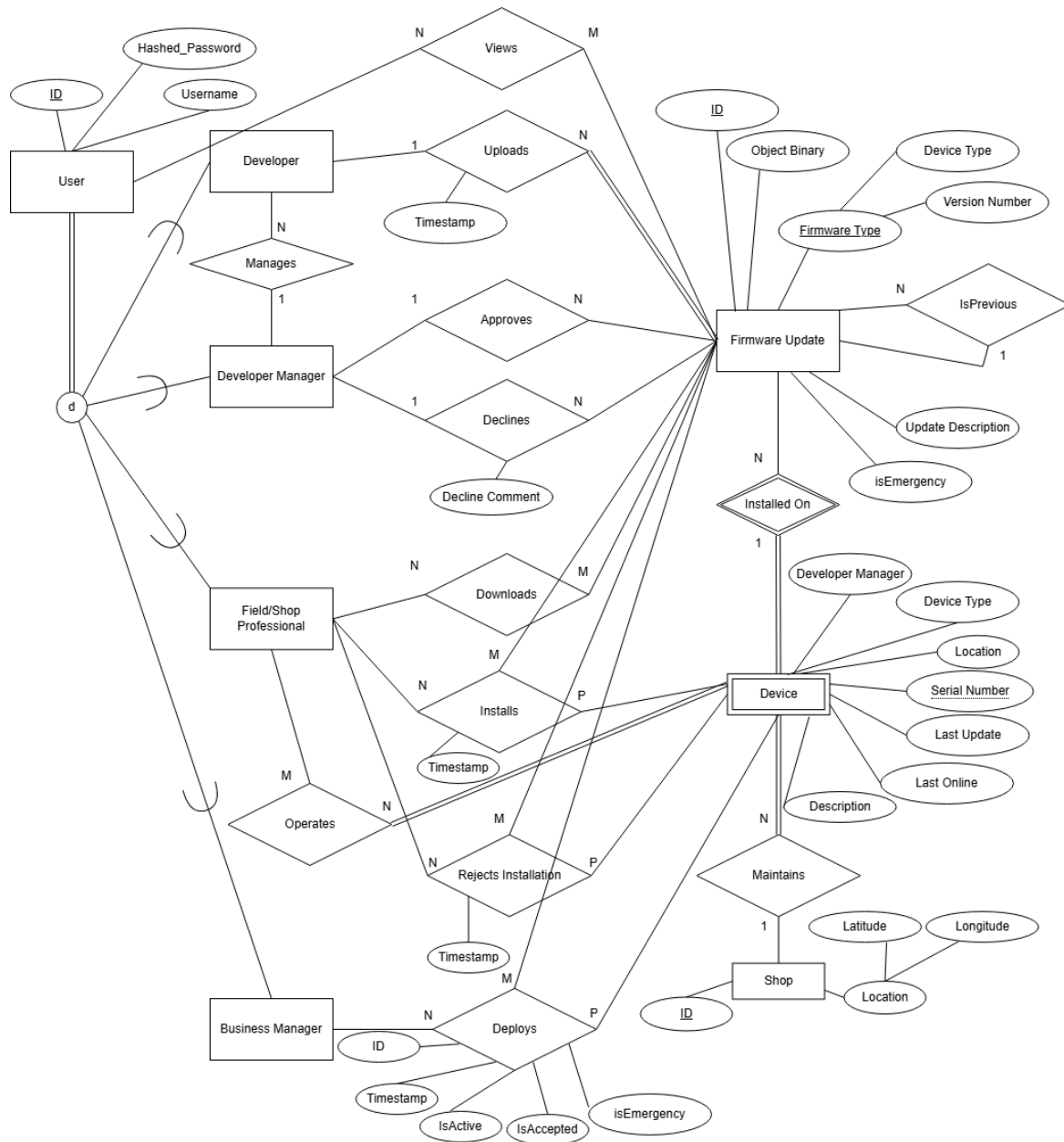


Figure 2 Entity Relationship Diagram

Here is a list of all major entities:

1. **Developer**: One of the roles of the user, who can upload firmware updates. The developer is managed by the developer manager.
2. **Developer Manager**: One of the roles of the user, who can approve or decline firmware updates and manage developers.

3. Field/Shop Professional: One of the roles of the user, who can download firmware updates to local devices or install the firmware update to the device or reject firmware update installation.
4. Business Manager: One of the roles of the user, who can deploy the firmware updates to the devices.
5. Firmware Updates: Contains the information about firmware type, update description from the developer, and a Boolean flag indicating if the update is an emergency or not. Firmware updates are installed on devices.
6. Devices: Contains the location of the devices, timestamps of last update and last online, and the description of the devices.

File Use

Uploaded firmware will be stored as compiled, binary files zipped together into a zip file. These files will be stored in Azure Blob Storage.

Data Exchange

Basic Transport Layer Security (TLS) will be utilized in communication between Azure IoT Hub and Edge Devices. Specifically, we will be employing the X.509 to format key certificates for our TLS.

Security Considerations

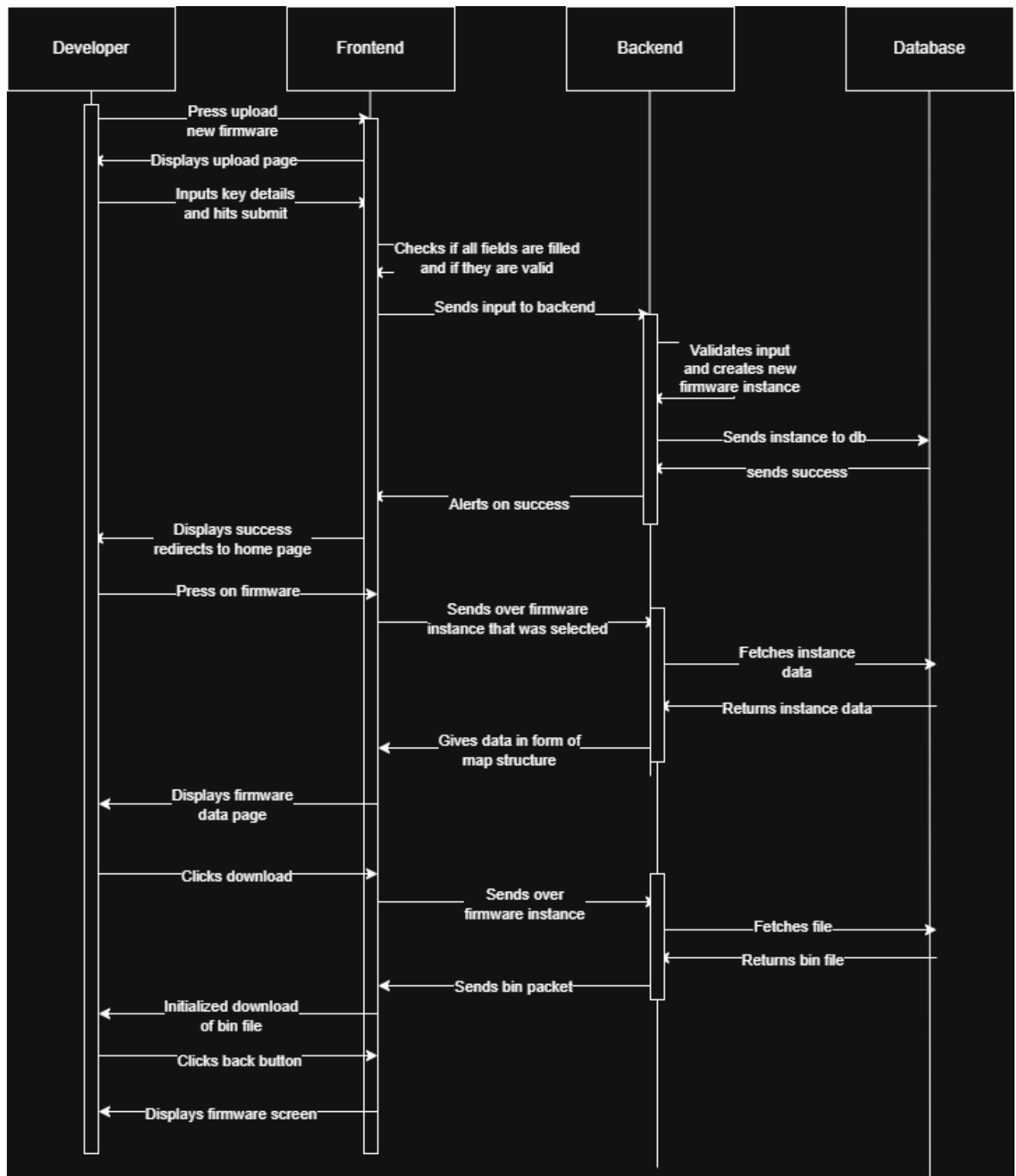
Malicious users can hijack our network packets and try to inject their own binary files into our channels. This is why we need basic protection. Furthermore, data encryption is needed for the log in information as well as the database data. TLS will be used to ensure secure connections when exchanging data across channels. User Login is required for this application. Users will login by entering a username and password. Passwords are hashed in database for protection. We do not work with any financial data on this project, and the only PII in our application would be the names of the developers of certain firmware. This keeps the information safe since the only other users that may see the developer's name would be the developer manager who is already in charge of the developer. The biggest privacy concern at the moment is ensuring that user login is not only being hashed in our database, but this login data should also be salted on top of being hashed to ensure that password multiple of the same password do not provide hash when being stored in our database.

Dynamic System Architecture

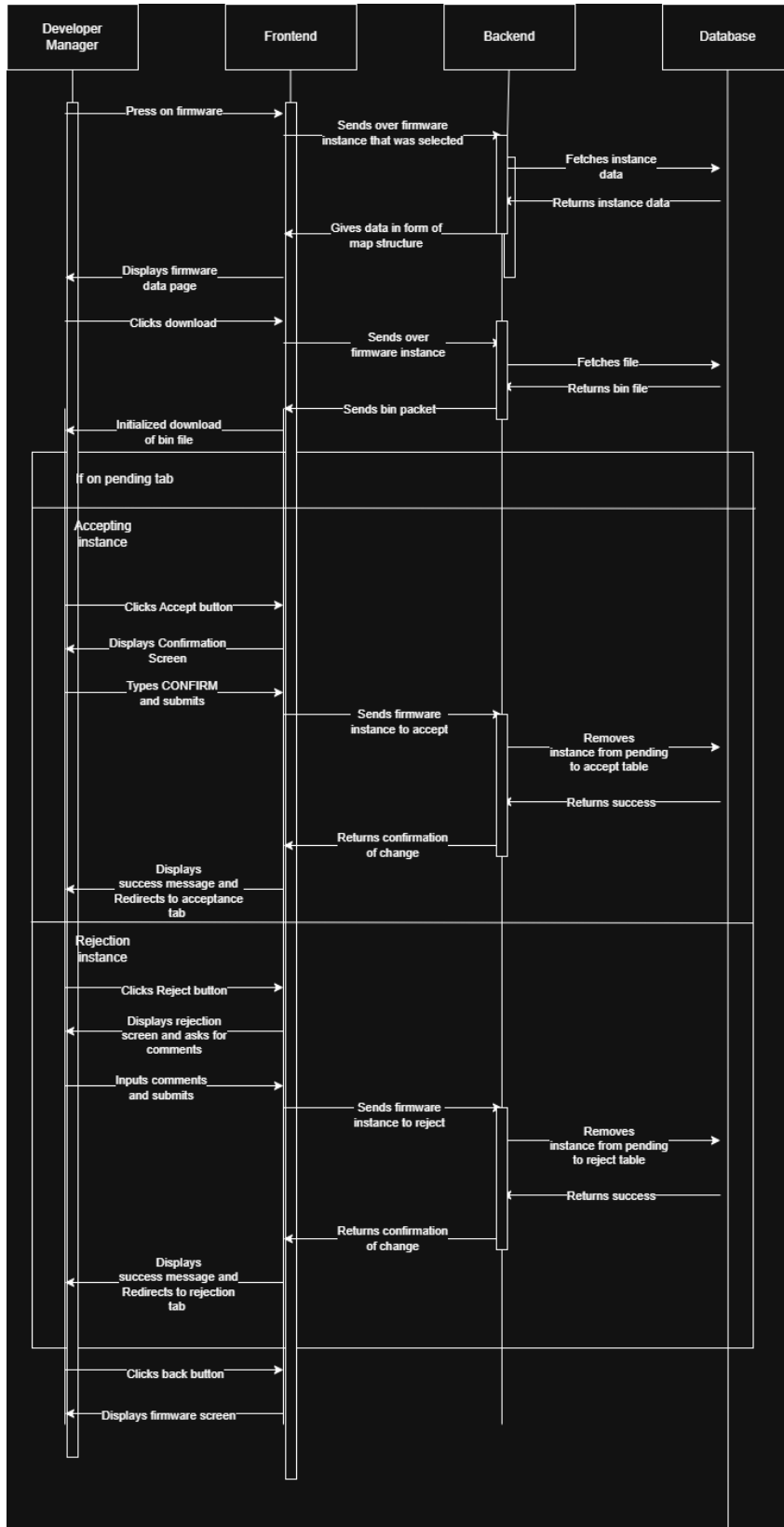
Introduction

In the following portion of the detailed design report, we will go into depth about all the user flows that we have built. To recap, we have four user flows. Developers, who upload firmware, Developer Managers, who review and approve firmware, Business Managers, who deploy firmware to devices, and Field Technicians, who accept deployments and install firmware on edge devices. We will sequentially go through every user flow and how they interact with our System Architecture through all the non trivial tasks each user is meant to accomplish.

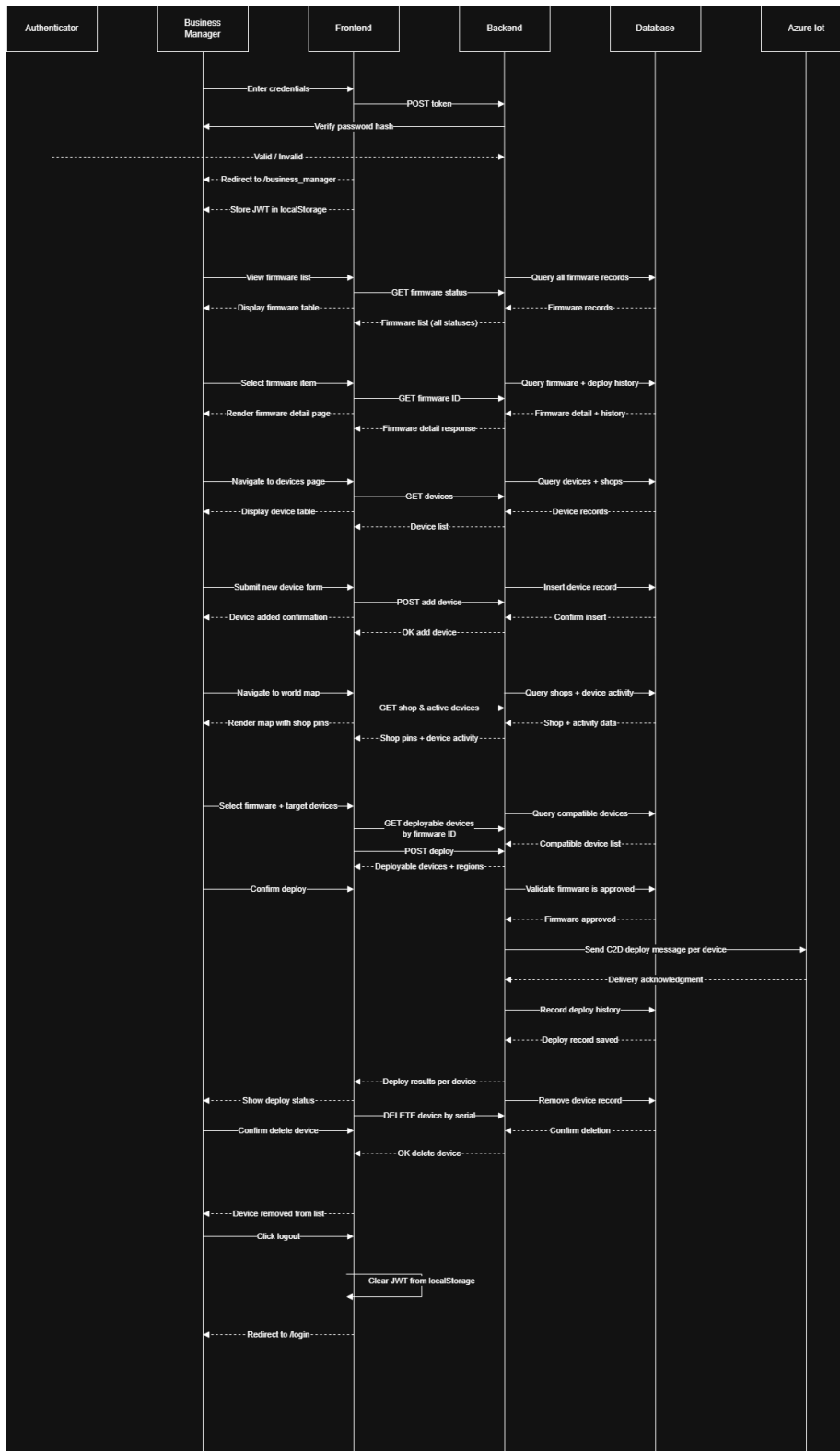
Developer



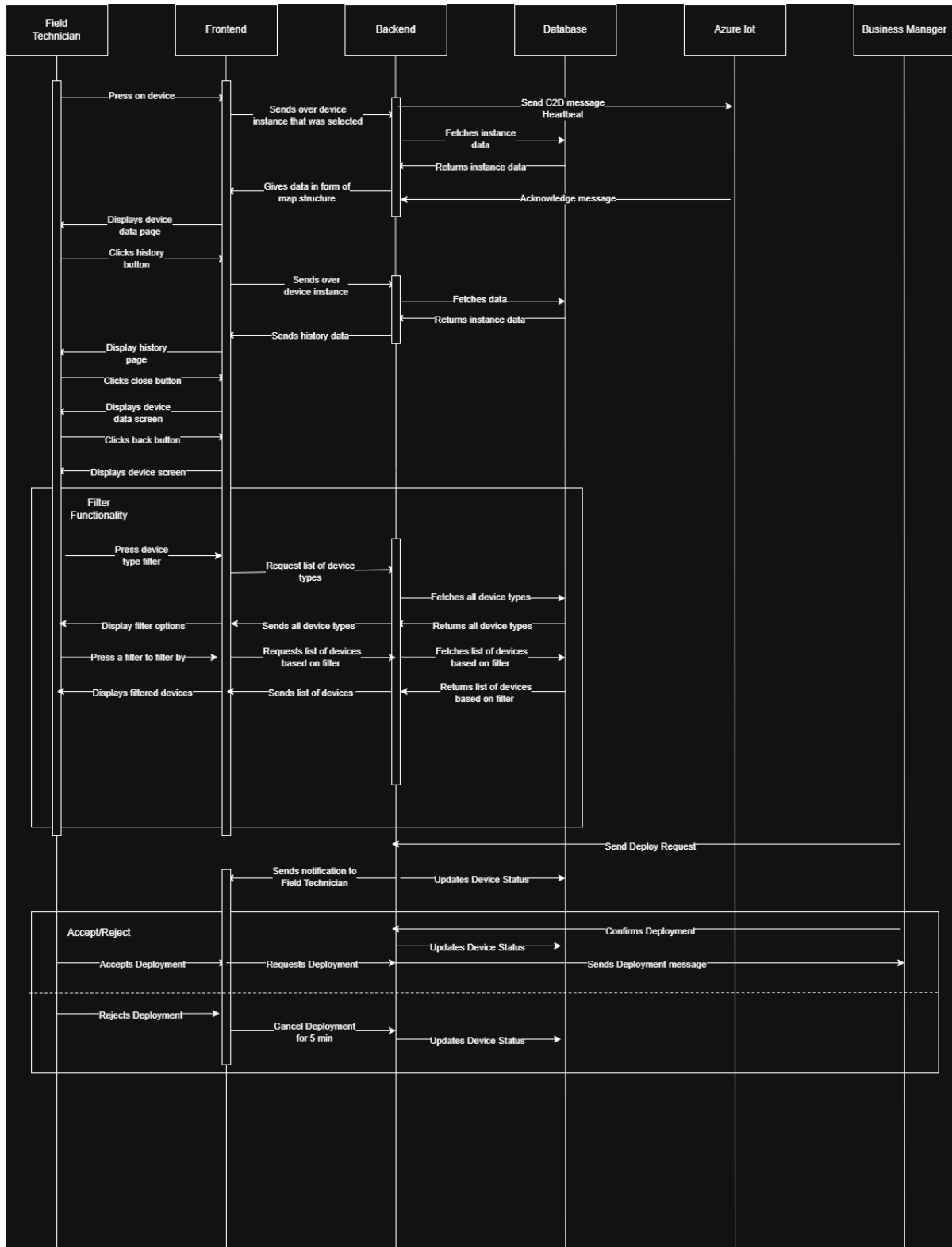
Developer Manager



Business Manager



Field Technician



UI Design and Heuristic Analysis

In this section, we demonstrate the user interface interaction and design of the application. We included all four views of the users: Developer, Developer Manager, Business Manager, Field Professional. The images are the major screens of the application, and the description below explains the interaction and heuristics that are applied.

Developer

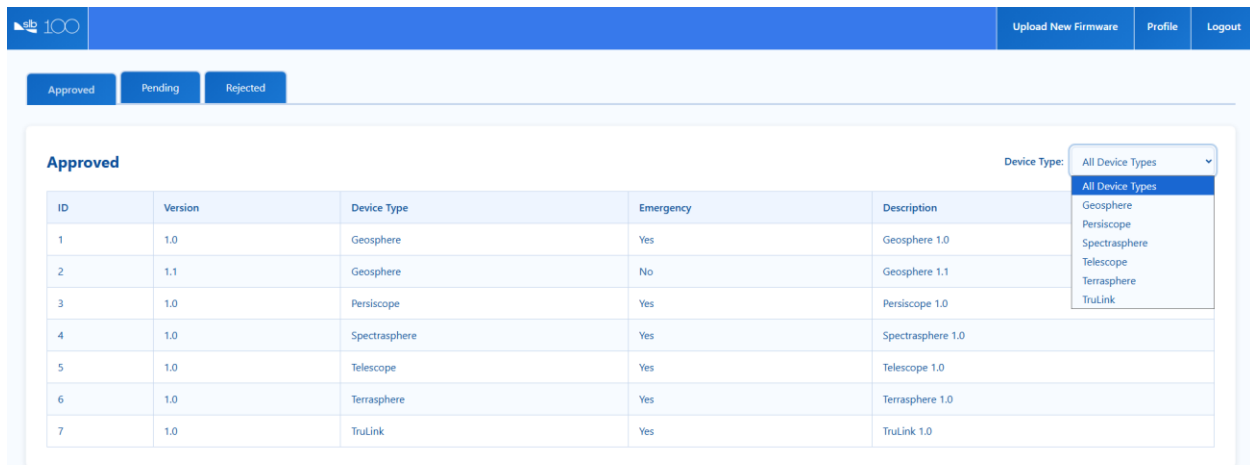


Figure 3 Developer: Firmware Page

Developers can see the firmware that they have access to. The firmware has three statuses: Approved, Pending, Rejected. We developed a table that distinguishes these statuses. By clicking Approved, Pending, Rejected buttons on the top of the table, developers can see corresponding tables. They can filter the device type by using a dropdown menu on the top right of the table.

Applied Heuristics:

Heuristic 1 Visibility of System Status is applied as the active tab is visible by bolded text and the button shifted down. It tells the users what table they are viewing.

Heuristic 6: Recognition Rather than Recall is applied as the dropdown filter for device type to make the options visible rather than requiring the users to remember the device type.

Heuristic 8: Minimalist Design is applied as the visual design focused on the essentials. There are no components that are unnecessary.

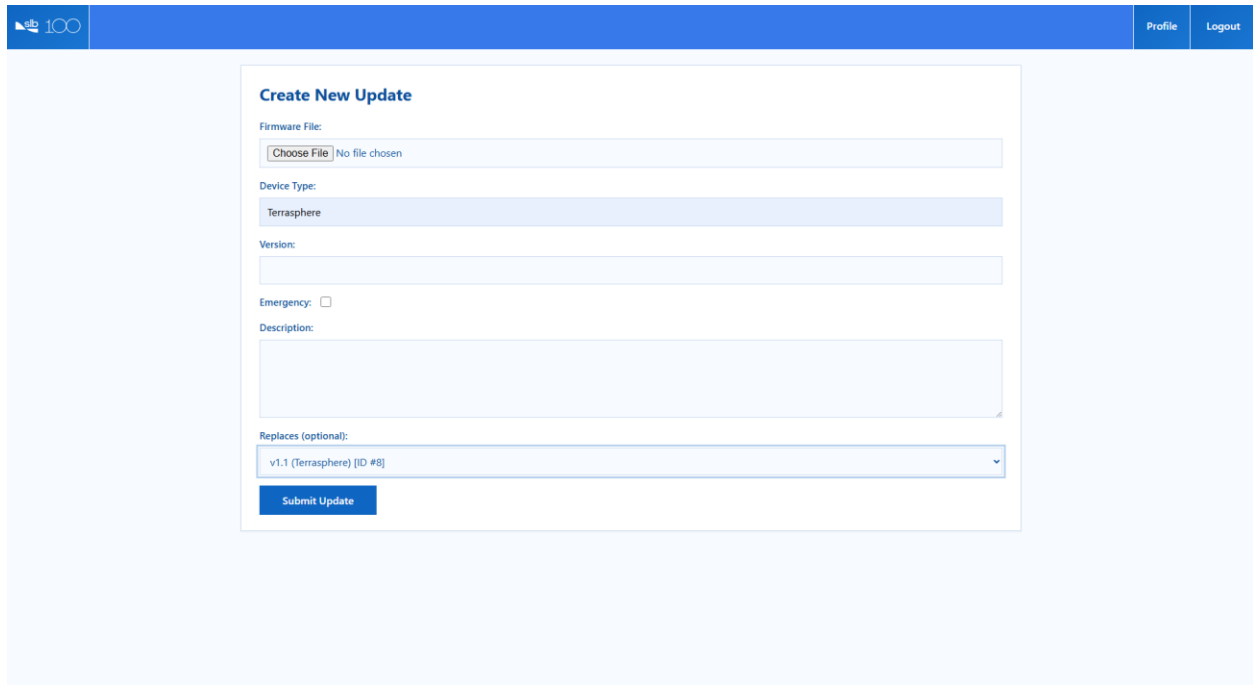
The image shows a web application interface with a blue header bar. On the left of the header is a logo with the text '100'. On the right are two links: 'Profile' and 'Logout'. Below the header is a white form titled 'Create New Update'. The form contains several input fields: 'Firmware File:' with a 'Choose File' button and 'No file chosen' text; 'Device Type:' with a dropdown menu showing 'Terrasphere'; 'Version:' with a text input field; 'Emergency:' with an unchecked checkbox; 'Description:' with a large text area; and 'Replaces (optional):' with a dropdown menu showing 'v1.1 (Terrasphere) [ID #8]'. At the bottom of the form is a blue 'Submit Update' button.

Figure 4 Developer: Upload New Firmware

Developers can create a new firmware update. From the previous image, they can click the “Upload New Firmware” button to navigate to this screen. They can upload firmware file from their machine, and input device type, version number, whether it is an emergency update or not, description of the update, and if this replaces previous rejected updates. After putting all the information, they can click the “Submit Update” button to upload.

Applied Heuristics:

Heuristic 4: Consistency and Standards is applied as the form uses a consistent layout that each field presents its label above the input.

Heuristic 6: Recognition Rather than Recall is applied as the dropdown menu for "Replaces" shows existing firmware entries to the user so that the users don't have to remember to look up version number.

Heuristic 8: Aesthetic and Minimalist Design is applied as the form consisting of essential input without any distracting unnecessary components.

Developer Manager

The screenshot shows a 'Firmware Details' form with a table of fields and a 'Prior Rejection History' section. The table contains the following data:

ID	9
Version	1.11
Device Type	Terrasphere
Emergency	No
Description	Terrasphere 1.11
Status	pending
Uploaded By	expodev
Upload Timestamp	2026-05-01T20:55:11.185679
Approved By	-
Declined By	-
Decline Comment	-

Below the table is a 'Prior Rejection History' section with the following text:

v1.1 (ID #8) — 5/1/2026
Rejected by: expodevimg
Reason: This firmware needs more functions.

At the bottom of the form are two buttons: 'Accept' and 'Reject'.

Figure 5 Developer Manager: Firmware Details

Developer Managers also have access to the firmware that they manage. The core job for them is to review the uploaded firmware from the developers that they manage and accept or decline the update. By clicking the row item from the table, they can view the detail for the firmware. Along with what the developer inputted as the firmware description, this page shows who approved or declined, and the decline comment from Developer Manager if declined. It also shows if this firmware replaces prior rejected firmware by showing the version and rejection reason.

Applied Heuristics:

Heuristic 3: User Control and Freedom is applied as the “Back” button clearly indicates the exit to go back to the previous screen.

Heuristic 6: Recognition Rather than Recall is applied as the Prior Rejection History delivers to the user that this certain version was rejected, and this new firmware tries to solve the problem. Users do not need to go back to the rejected table to search for the reason for the new upload.

Heuristic 8: Aesthetic and Minimalist Design is applied as the page only has essential information about the specific firmware, and there are no unnecessary components.

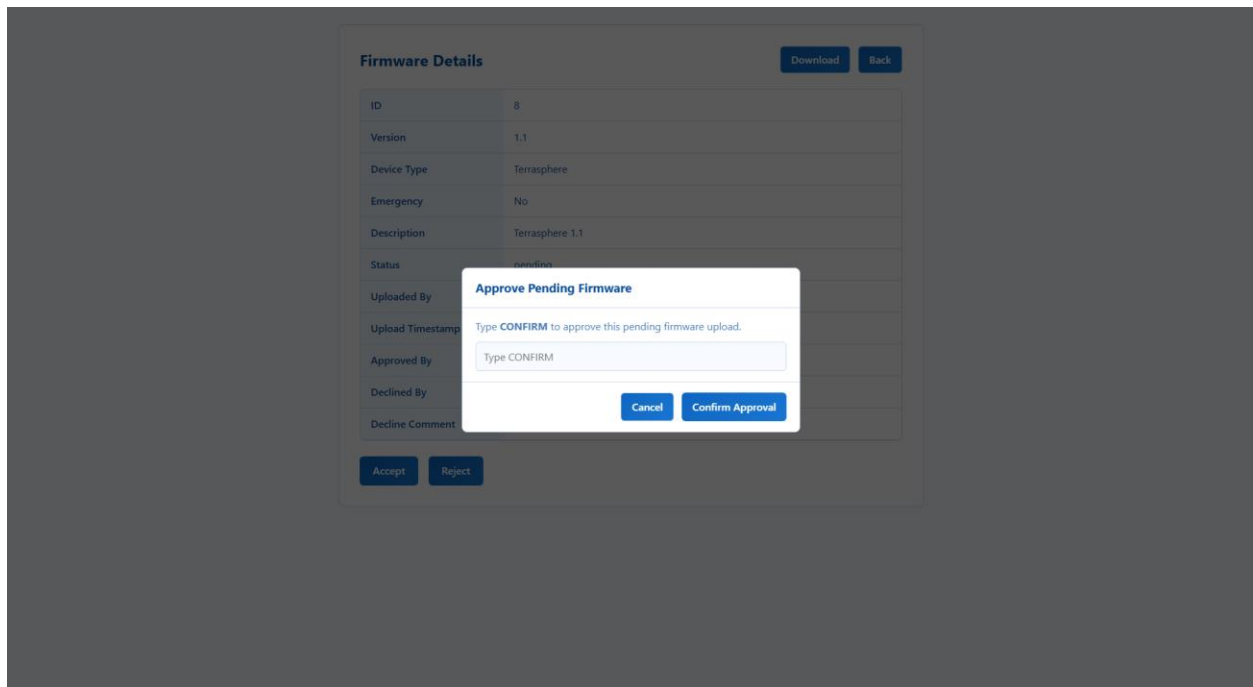


Figure 6 Developer Manager: Approve Pending Firmware

Developer Managers can accept the firmware by clicking the “Accept” button on the bottom left. As they click, the confirmation box pops up and requires them to type “confirm” before accepting.

Applied Heuristics:

Heuristic 3: User Control and Freedom is applied as the “Cancel” button clearly indicates to cancel approving.

Heuristic 5: Error Prevention is applied as the user is required to type “confirm” to complete approving. This prevents instant acceptance by mis-clicking the “Accept” button.

Heuristic 8: Aesthetic and Minimalist Design is applied as the page only has essential components to type “confirm” to continue and press “Cancel” to cancel the approval. There are no unnecessary components.

The image shows a 'Reject Pending Firmware' dialog box. It contains the following fields and values:

- Firmware ID: 9
- Version: 1.11
- Device Type: Terrasphere
- Emergency: No
- Description: Terrasphere 1.11
- Current Status: pending
- Rejecting Developer Manager: expodevmng
- Reason for Rejection: Provide rejection reason

At the bottom right, there are two buttons: 'Cancel' and 'Confirm Rejection'.

Figure 7 Developer Manager: Reject Pending Firmware

Developer Managers can reject the firmware by clicking the “Reject” button next to the “Accept” button. Then, the rejection box pops up and shows the firmware details to review and on the very bottom, there is a place to input rejection reason. They can click the “Confirm Rejection” button to complete rejection.

Applied Heuristics:

Heuristic 3: User Control and Freedom is applied as the “Cancel” button clearly indicates to cancel rejection process.

Heuristic 5: Error Prevention is applied as the user is required to provide rejection reason to complete rejection. This prevents instant acceptance by mis-clicking the “Reject” button.

Heuristic 6: Recognition Rather than Recall is applied as the page provides the details of the firmware to review, so that the users do not have to memorize the details or go back to look at the details.

Business Manager

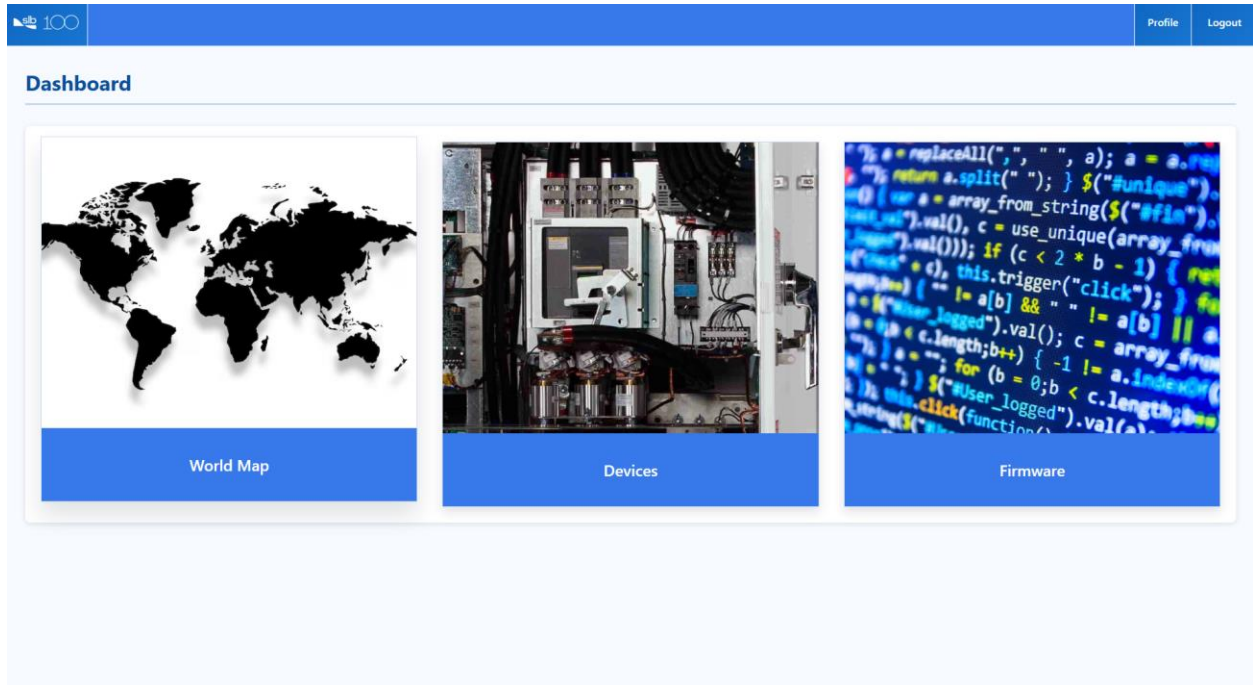


Figure 8 Business Manager: Home Page

This image is the home page for the Business Managers. Since they have access to all the devices and firmware, and the map to view the status of devices, there are three buttons representing each functionality. Clicking the “World Map” button navigates to the map page, “Devices” button navigates to the devices page, and “Firmware” button navigates to the firmware page.

Applied Heuristics:

Heuristic 1: Visibility of System Status is applied since as the user hovers over the button, the button shifts upward, and the shadow appears to show that the user is hovering over the button.

Heuristic 4: Consistency and Standards is applied as the navigation cards share a consistent layout. The image is on the top, and the text is on the bottom.

Heuristic 8: Aesthetic and Minimalist Design is applied as the three navigation cards are the main components of the page, and there are no other distracting unnecessary components.

World Map

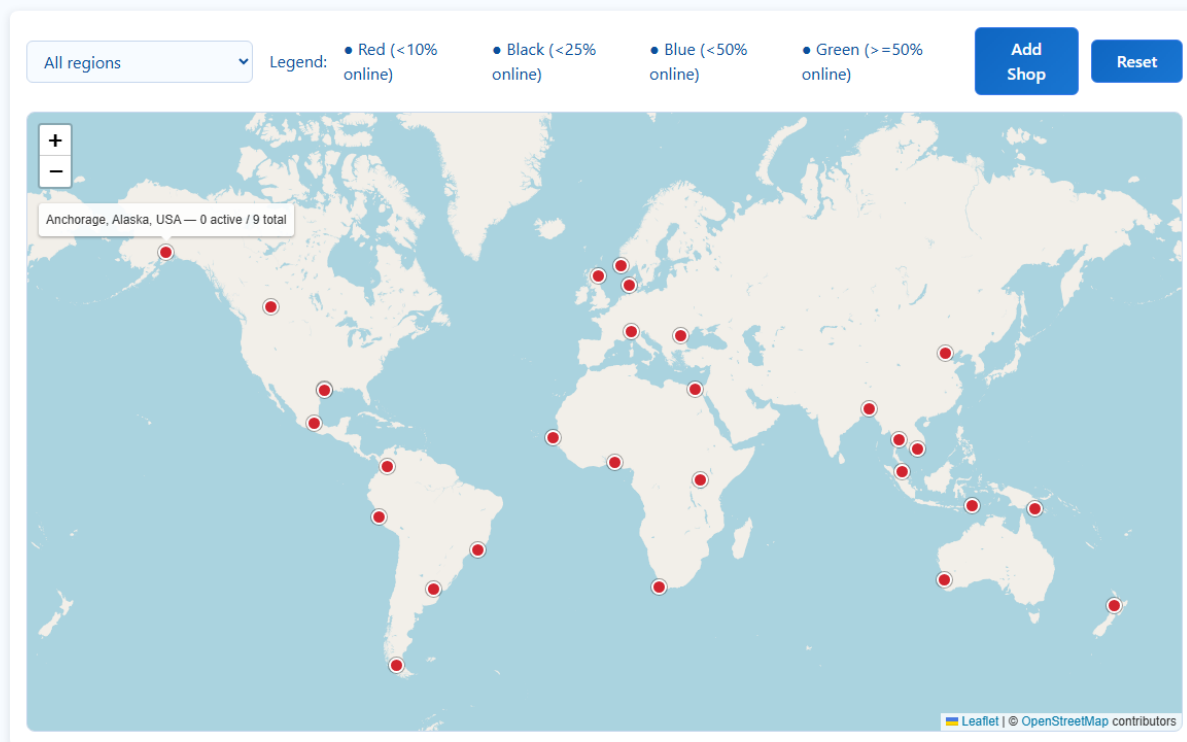


Figure 9 Business Manager: World Map

After clicking the “World Map” button, business managers will be navigated to this world map page. This image shows the world map and dots across the globe. Each dot represents the maintenance shops. They can hover over the dots to see the location name of the shop and see how many are active out of all the devices.

Applied Heuristics:

Heuristic 1: Visibility of System Status is applied as the tooltip on hover gives the user the information of the status of the active/inactive devices. This informs the users what is going on with the devices.

Heuristic 3: User Control and Freedom is applied as the reset button resets the location of the map, so that when the users mistakenly zoom in too much and lose their location, they can reset it and start again.

Heuristic 6: Recognition Rather than Recall is applied as the legend at the top makes the color coding visible so that the users do not need to memorize what each color means.

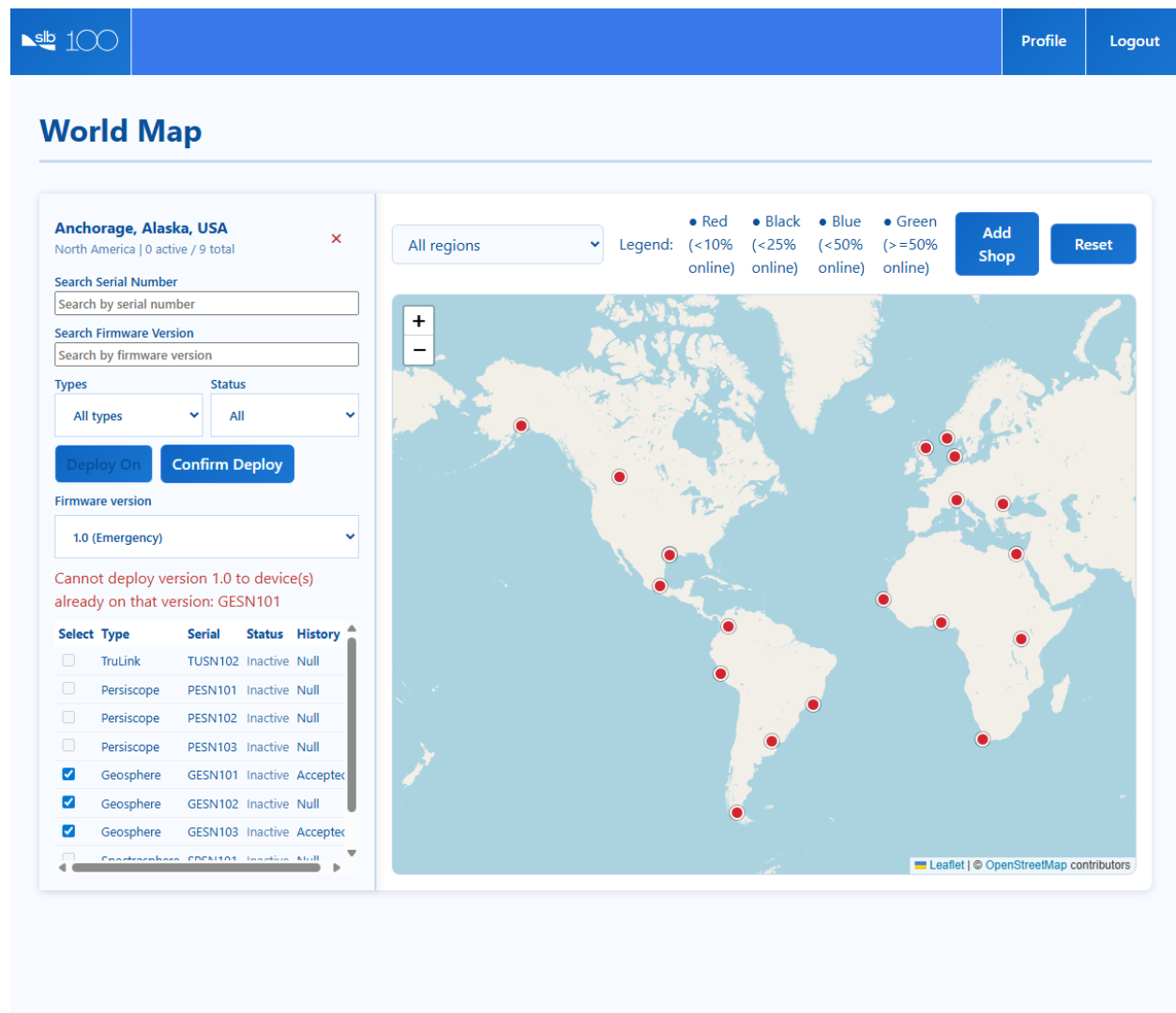


Figure 10 Business Manager: World Map Panel

By clicking the dots, the panel that shows devices information slides in. From this panel, the business managers can see what devices are maintained from this shop and active status and recent acceptance status of the firmware. They can toggle the “Deploy” button to deploy firmware to the device. When it is clicked and in the deploy mode, they can select multiple same devices and choose the firmware version from the dropdown menu

to deploy. If the selected firmware cannot be deployed to the device, it gives feedback on which firmware prevents multi-deployment.

Applied Heuristics:

Heuristic 1: Visibility of System Status is applied as when the deployment is prevented; it gives the error status feedback of why it was prevented. This quick feedback informs the user what is going on.

Heuristic 3: User Control and Freedom is applied as the “x” button in the top right of the panel closes the panel and lets the users view the full view of the map.

Heuristic 6: Recognition Rather than Recall is applied as the device information is provided in the panel. Users do not need to remember the serial number or navigate to the device page to check the serial number.

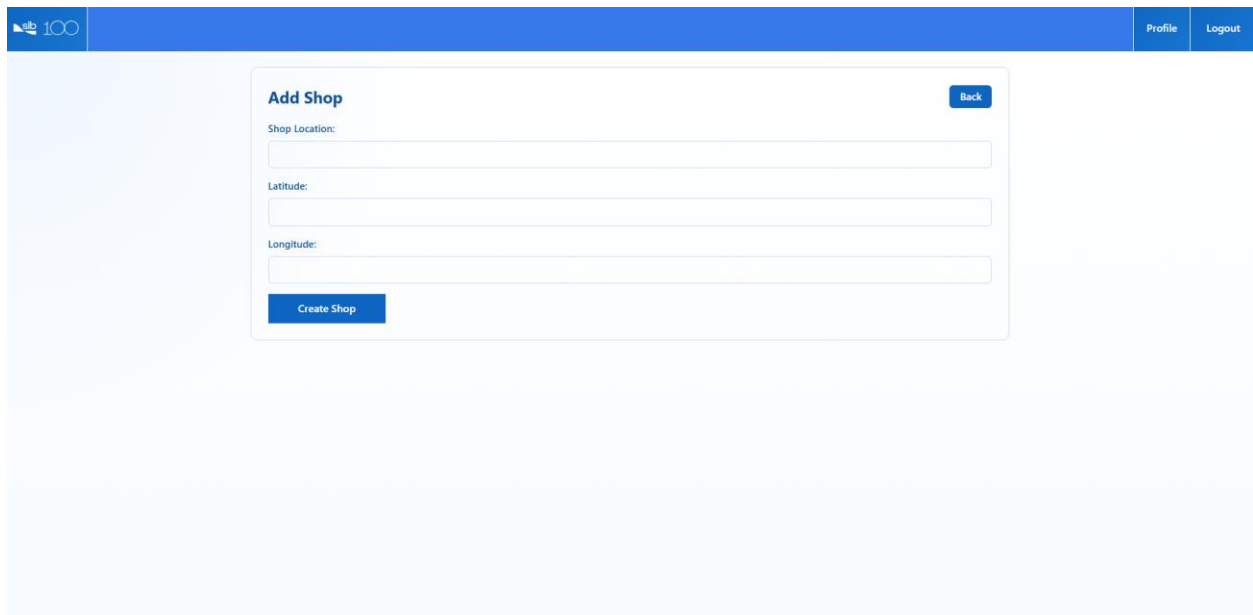
The image shows a web application interface for 'Business Manager'. At the top, there is a blue header bar with a logo on the left and 'Profile' and 'Logout' links on the right. Below the header, the main content area is light blue. In the center, there is a white rectangular panel titled 'Add Shop' in blue text. Inside this panel, there are three input fields labeled 'Shop Location:', 'Latitude:', and 'Longitude:'. Below these fields is a blue button labeled 'Create Shop'. In the top right corner of the 'Add Shop' panel, there is a small blue button labeled 'Back'.

Figure 11 Business Manager: Add Shop

Business managers can create shops. They can input the name of the location, latitude, and longitude to put a pin on the map.

Applied Heuristics:

Heuristic 3: User Control and Freedom is applied as the “Back” button clearly indicates that the users can navigate back to the previous screen.

Heuristic 4: Consistency and Standards is applied as the layout of the form is consistent, maintaining the design consistency.

Heuristic 8: Aesthetic and Minimalist Design is applied as the page consists of components only for adding shop functionality.

Devices						
All Active Devices						
Device Type	Firmware Version	Last Updated	Region	Serial Number	Device Description	Remove
Persiscope	N/A	2026-04-27 04:02	Kampala, Uganda	PESN104	Multilayer mapping-while-drilling service	X
Telescope	N/A	2026-04-27 04:03	Kampala, Uganda	TESN104	High-speed telemetry-while-drilling service	X
TruLink	1.0	2026-04-27 13:41	Kampala, Uganda	TUSN103	High-definition wellbore architecture from tophole to TD	X
TruLink	1.0	2026-04-27 13:41	Beijing, China	TUSN104	High-definition wellbore architecture from tophole to TD	X
TruLink	N/A	2026-04-27 04:09	Anchorage, Alaska, USA	TUSN102	High-definition wellbore architecture from tophole to TD	X
Terrasphere	1.0	2026-04-27 05:07	Beijing, China	TRSN102	High-definition dual-imaging-while-drilling service	X
Persiscope	N/A	2026-04-27 04:22	Anchorage, Alaska, USA	PESN101	Multilayer mapping-while-drilling service	X
Persiscope	N/A	2026-04-27 04:22	Anchorage, Alaska, USA	PESN102	Multilayer mapping-while-drilling service	X
Persiscope	N/A	2026-04-27 04:22	Anchorage, Alaska, USA	PESN103	Multilayer mapping-while-drilling service	X
Geosphere	1.0	2026-04-27 16:39	Anchorage, Alaska, USA	GESN101	3D reservoir mapping-while-drilling service	X
Geosphere	N/A	2026-04-27 04:37	Anchorage, Alaska, USA	GESN102	3D reservoir mapping-while-drilling service	X
Geosphere	1.1	2026-04-27 16:30	Anchorage, Alaska, USA	GESN103	3D reservoir mapping-while-drilling service	X
Spectrasphere	N/A	2026-04-27 05:20	Anchorage, Alaska, USA	SPSN101	Fluid mapping-while-drilling service	X

Figure 12 Business Manager: Devices Page

The devices page for business managers is also a table view. The table contains device type, firmware version, last updated time, name of the region, unique serial number, device description and a remove button.

Applied Heuristics:

Heuristic 2: Match Between the System and the Real World is applied as the column labels are not internal jargon but actual real-world conventions.

Heuristic 3: User Control and Freedom is applied as the users can remove devices by clicking the “x” button on the right when they mistakenly create devices with wrong information.

Heuristic 4: Consistency and Standards is applied as the layout of the table is consistent, maintaining the design consistency.

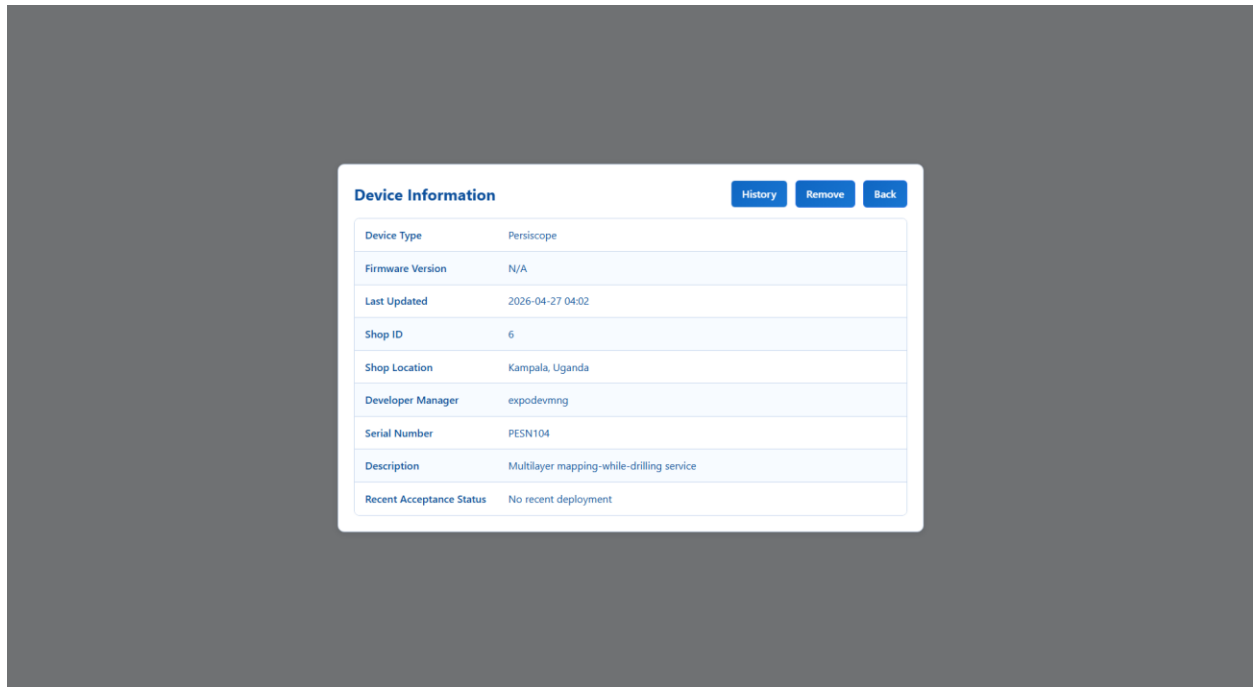


Figure 13 Business Manager: Device Information

Similar to the firmware detail page, business managers can click the row item of the table to view the detailed information of the devices. Along with the information from the table, it shows the recent acceptance status.

Applied Heuristics:

Heuristic 3: User Control and Freedom is applied as the “Back” button clearly indicates that the users can navigate back to the previous screen.

Heuristic 4: Consistency and Standards is applied as the two-column layout of detailed information page is consistent with firmware detailed page. So that it is easier for the users to read and understand.

Heuristic 6: Recognition Rather than Recall is applied as all device information is provided in one screen, so the users do not need to navigate to another page to look up the information.

Deploy ID	Firmware Version	Timestamp	Status	Acceptance Status
43	1.0	2026-04-27 16:39	Active	Accepted
42	1.1	2026-04-27 16:34	Inactive	Accepted
41	1.0	2026-04-27 16:32	Inactive	Accepted
40	1.0	2026-04-27 16:30	Inactive	Accepted
38	1.1	2026-04-27 16:25	Inactive	Accepted
36	1.1	2026-04-27 16:25	Inactive	Pending
34	1.0	2026-04-27 16:03	Inactive	Accepted
32	1.1	2026-04-27 15:45	Inactive	Accepted
30	1.0	2026-04-27 15:43	Inactive	Accepted
28	1.1	2026-04-27 15:17	Inactive	Accepted
26	1.0	2026-04-27 15:11	Inactive	Accepted
24	1.0	2026-04-27 15:07	Inactive	Accepted
22	1.1	2026-04-27 15:01	Inactive	Accepted
20	1.0	2026-04-27 14:51	Inactive	Accepted

Figure 14 Business Manager: Deployment History

From the previous screen, business managers can view the deploy history by clicking the “History” button on the top right of the box. This history page shows deployment ID, firmware version, time of the deployment, active status, and acceptance status.

Applied Heuristics:

Heuristic 1: Visibility of System Status is applied as the table shows what firmware is active on this device and the acceptance status from the field professionals.

Heuristic 3: User Control and Freedom is applied as the “Close” button clearly indicates to close the tab and show the previous tab.

Heuristic 4: Consistency and Standards is applied as layout of the table is consistent with firmware and device tables. This maintains the design consistency.

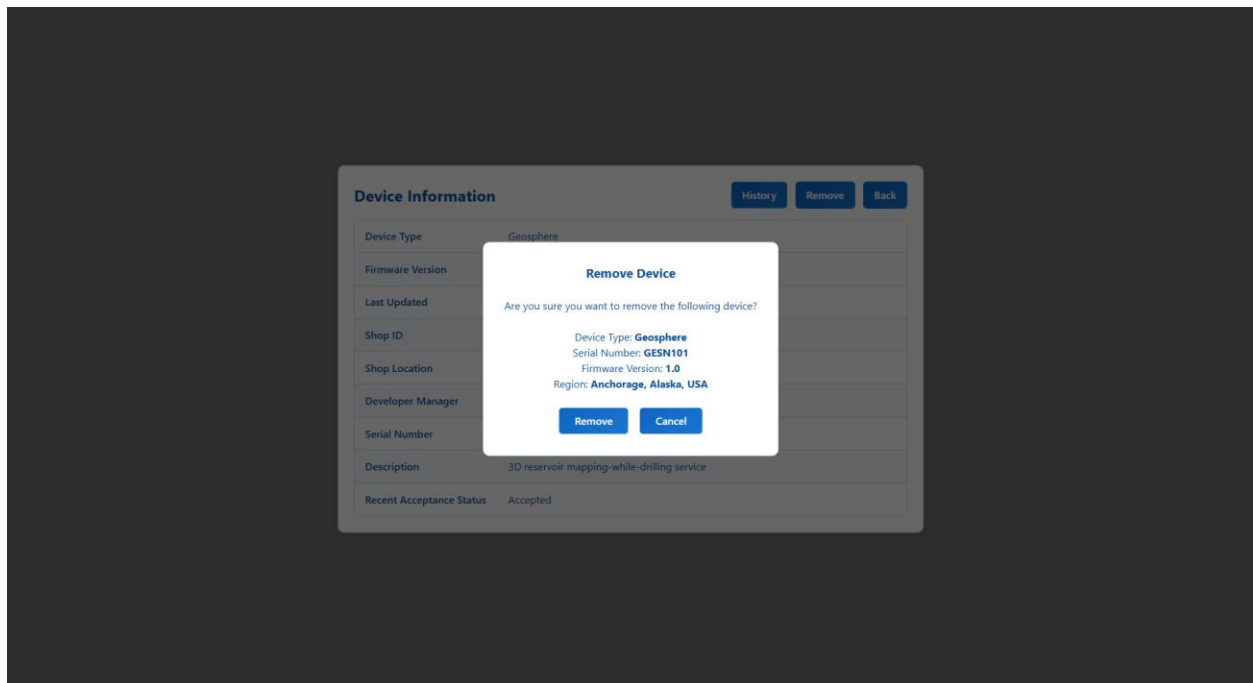


Figure 15 Business Manager: Remove Device

When business managers click the remove button, the confirmation screen pops up. It shows the device information to review. They can complete removing by clicking the “Remove” button or cancel removing by clicking the “Cancel” button.

Applied Heuristics:

Heuristic 3: User Control and Freedom is applied as the “Cancel” button clearly indicates that it would cancel the removing process.

Heuristic 5: Error Prevention is applied as the pop up reassures the users that they want to remove the device. So, if the user mistakenly clicks the “Remove” button, it will not immediately remove the device.

Heuristic 6: Recognition Rather than Recall is applied as it shows the users what device they are about to remove, so that the users do not have to remember all the details about the device when they are confirming the removal.

The screenshot shows a web interface for adding a new device. At the top, there is a blue header bar with a logo on the left and 'Profile' and 'Logout' links on the right. The main content area is white and contains a form titled 'Add New Device'. The form has the following fields:

- Device Type: A text input field.
- Serial Number: A text input field.
- Shop: A dropdown menu with the placeholder text 'Select a Shop'.
- Developer Manager: A dropdown menu with the placeholder text 'Select a Developer Manager'.
- Field Shop Professionals: A dropdown menu with the placeholder text 'Select a Field Shop Professional'.
- Device Description: A large text area.

At the bottom right of the form is a blue button labeled 'Add New Device'. Below the form, a red error message reads: 'All fields are required. Please fill in every field before submitting.'

Figure 16 Business Manager: Add New Device

To add the device, business managers click the “Add New Device” button on top right. They can input device type, serial number, device description and select shop, developer manager, field shop professionals. They can complete the process by clicking the “Add New Device” button. If any fields are not filled, it gives feedback to fill in every field.

Applied Heuristics:

Heuristic 4: Consistency and Standards is applied as the layout of the form is consistent, maintaining the design consistency.

Heuristic 6: Recognition Rather than Recall is applied as the dropdown menus for selecting developer manager and field shop professionals provide the options, so that the users do not have to memorize developer manager and field shop professionals’ usernames.

Heuristic 9: Help Users Recognize, Diagnose, and Recover from Errors is applied as the red error message indicates the problem that some field is missing. So that the user can easily recover from errors of missing values.

100				Profile	Logout
Approved Pending Rejected					
Approved				Device Type: All Device Types	
ID	Version	Device Type	Emergency	Description	
1	1.0	Geosphere	Yes	Geosphere 1.0	
2	1.1	Geosphere	No	Geosphere 1.1	
3	1.0	Persiscope	Yes	Persiscope 1.0	
4	1.0	Spectrasphere	Yes	Spectrasphere 1.0	
5	1.0	Telescope	Yes	Telescope 1.0	
6	1.0	Terrasphere	Yes	Terrasphere 1.0	
7	1.0	TruLink	Yes	TruLink 1.0	

Figure 17 Business Manager: Firmware Page

Clicking the “Firmware” button will navigate business managers to the firmware page, which is the same view as developer managers, but business managers can view all the firmware.

Applied Heuristics:

Heuristic 1 Visibility of System Status is applied as the active tab is visible by bold text and the button shifted down. It tells the users what table they are viewing.

Heuristic 6: Recognition Rather than Recall is applied as the dropdown filter for device type to make the options visible rather than requiring the users to remember the device type.

Heuristic 8: Minimalist Design is applied as the visual design focused on the essentials. There are no components that are unnecessary.

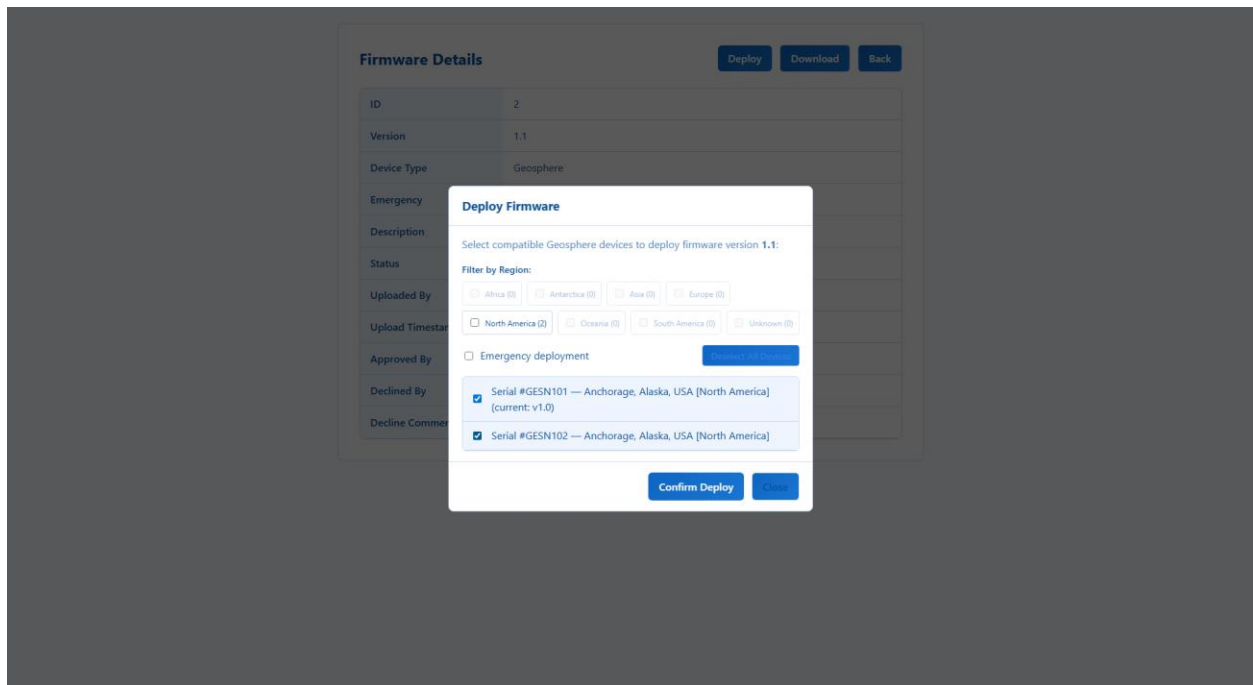


Figure 18 Business Manager: Deploy Firmware

The firmware detail page for business manager has a deploy button on the top right. When it is clicked, the deploy firmware box pops up. From this box, they can filter by the region to view the devices in certain regions. They can make the deployment an emergency by clicking the emergency deployment checkbox. They will select devices and complete deployment by clicking the “Confirm Deploy” button.

Applied Heuristics:

Heuristic 3: User Control and Freedom is applied as the “Close” button clearly indicates to close the tab and show the previous tab.

Heuristic 6: Recognition Rather than Recall is applied as the selected devices show the current firmware version. So the users do not need to memorize the version or navigate to another page to view the version.

Heuristic 7: Flexibility and Efficiency of Use is applied as filtering by region checkbox may speed up the device searching time and “Select All Devices” and “Deselect All Devices” will speed up selecting the devices.

Field Professional

100					Profile	Logout
My Assigned Devices						
Devices					Device Type: All Device Types	
Device Type	Firmware Version	Last Updated	Location	Serial Number	Description	
Geosphere	1.0	2026-04-27 16:39	Anchorage, Alaska, USA	GESN101	3D reservoir mapping-while-drilling service	
Geosphere	N/A	2026-04-27 04:37	Anchorage, Alaska, USA	GESN102	3D reservoir mapping-while-drilling service	
Geosphere	1.1	2026-04-27 16:30	Anchorage, Alaska, USA	GESN103	3D reservoir mapping-while-drilling service	
Persiscope	N/A	2026-04-27 04:22	Anchorage, Alaska, USA	PESN101	Multilayer mapping-while-drilling service	
Persiscope	N/A	2026-04-27 04:22	Anchorage, Alaska, USA	PESN103	Multilayer mapping-while-drilling service	
Spectrasphere	N/A	2026-04-27 05:20	Anchorage, Alaska, USA	SPSN101	Fluid mapping-while-drilling service	
Spectrasphere	N/A	2026-04-27 05:21	Anchorage, Alaska, USA	SPSN102	Fluid mapping-while-drilling service	
TruLink	N/A	2026-04-27 04:09	Anchorage, Alaska, USA	TUSN102	High-definition wellbore architecture from tophole to TD	

Figure 19 Field Professional: View Devices

Field professionals can only view their assigned device. This page is also a table showing the devices. They are able to filter the table by device type using the dropdown menu on the top right of the table.

Applied Heuristics:

Heuristic 1 Visibility of System Status is applied as the active tab is visible by bold text and the button shifted down. It tells the users what table they are viewing.

Heuristic 6: Recognition Rather than Recall is applied as the dropdown filter for device type to make the options visible rather than requiring the users to remember the device type.

Heuristic 8: Minimalist Design is applied as the visual design focused on the essentials. There are no components that are unnecessary.

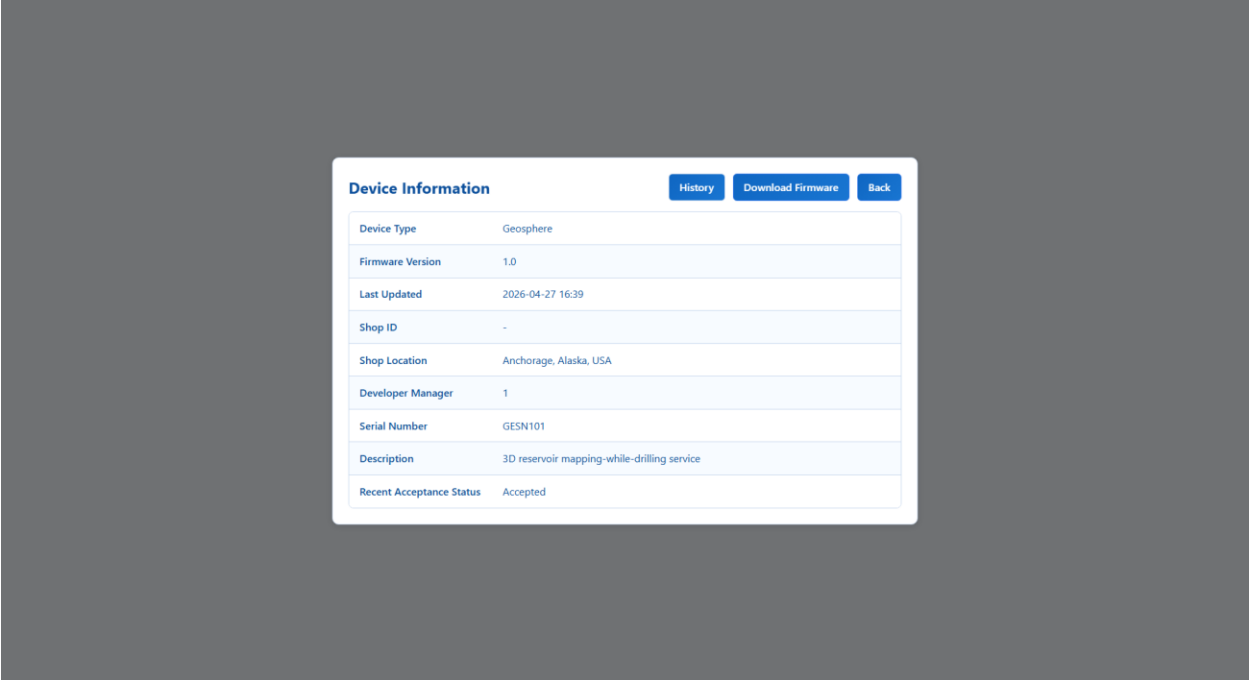


Figure 20 Field Professional: Device information

Similar to previous tables, field professionals can select an item from the table to view the details of the device. They can view the history by clicking the “History” button, download firmware by clicking the “Download Firmware” button. They can navigate to the previous screen by clicking the “Back” button.

Applied Heuristics:

Heuristic 3: User Control and Freedom is applied as the “Back” button clearly indicates that the users can navigate back to the previous screen.

Heuristic 4: Consistency and Standards is applied as the two-column layout of detailed information page is consistent with firmware detailed page. So that it is easier for the users to read and understand.

Heuristic 6: Recognition Rather than Recall is applied as all device information is provided in one screen, so the users do not need to navigate to another page to look up the information.

Appendix

Team Members:

Name	Contact Information	Contribution
------	---------------------	--------------

Yuvraj Dhadwal	ydhadwal3@gatech.edu	- Architected and implemented the control plane for edge devices - Database Setup - Azure IoT interfacing - Azure Cloud Web Hosting
Patrick Huynh	phuynh44@gatech.edu	- Developed developer firmware page, business manager home & device page, field shop professional device table - Assisted in firmware deployment to one or many devices - Implemented add/remove devices for business manager
Minjae Lee	mlee685@gatech.edu	- Database setup - Initial project setup - Acceptance status logic, Deployment history logic, Replace firmware rejection logic
Arthur Mahu	amahu3@gatech.edu	- UI Web Design - Business Manager Map Page View - Displaying online devices and shops - View Firmware Uploads
Christian Vela	cvela8@gatech.edu	- Upload and Download of Firmware from Website - Emergency Deployments - Redeploy Firmware after Rejection